



Time to read: 6–8 minutes

Topic goal: Understand how AI evolved from rule-based logic to foundation models—and why modern AI is primarily an **engineering** challenge.

Why history matters in an engineering course?

If you only learn today’s tools AI can feel like a collection of disconnected buzzwords: “deep learning”, “transformers”, “LLMs”, “MLOps”.

History turns that into a coherent story:

- **What changed over time?** Data, compute and model scale
- **What stayed consistent?** AI is used to improve decisions and automate reasoning.
- **Why does AI engineering matter today?** Because modern AI systems are large, data-driven and deployed into changing real-world environments

Milestone 1: Rule-Based Systems (Pre-2010)

What they were?

Early “AI” in industry was often **rule-based**:

- If condition A, then action B
- Logic handcrafted by domain experts

Examples

- Expert systems for diagnosis support
- Business rule engines (“if transaction > X and location changes, flag fraud”)
- Decision trees are manually designed for small workflows

Why they were small and deterministic?

Rule-based systems were limited by humans:

- Every new scenario required a new rule
- Rules were brittle and hard to maintain at scale
- They could not generalise beyond what humans anticipated

What they did well?

- Easy to test and debug (deterministic)
- Easy to explain (rules are transparent)
- Reliable in stable environments

Where they failed?

- Could not adapt to messy real-world patterns
- Collapsed when the number of exceptions grew
- Didn't improve automatically with more data

Key takeaway:

Rule-based AI was predictable—but limited. Human rule-writing was the bottleneck.

Milestone 2: Classical Machine Learning

What changed?

Classical ML replaced explicit rules with models that **learn patterns from data**.

Instead of coding rules, you provide:

- Examples (data)
- Correct answers (labels)
- A learning algorithm

Common methods

- Linear/logistic regression
- Decision trees and random forests
- Support Vector Machines

Why was it a big step?

Classical ML allowed systems to:

- Generalise from examples
- Handle variability better than rules
- Learn patterns humans didn't explicitly write

What still limited it?

- Feature engineering was mostly manual
- Data sizes were moderate
- Compute was constrained
- The models were still “small” compared to today

Key takeaway:

The “logic” began shifting from code to data—but systems were still manageable.

Milestone 3: Deep Learning (CNNs, RNNs)

What changed?

Deep learning introduced neural networks that could learn **representations** from raw data, reducing the need for handcrafted features.

Key breakthroughs

- **CNNs** (Convolutional Neural Networks): strong at vision tasks
- **RNNs / LSTMs**: better at sequential tasks (text, time-series)
- **GPUs**: enabled practical training on large datasets

Why deep learning mattered?

Deep learning unlocked significant improvements in:

- Image recognition
- Speech recognition
- Language processing

New challenges it introduced

As models grew larger:

- Training time increased (hours → days)
- Compute costs increased
- Debugging became harder (less interpretable)
- Deployment became harder (performance, latency, memory)

Key takeaway:

Deep learning improved capability—but also increased system complexity.

Milestone 4: Transformers and Attention

What the transformer changed?

Transformers, driven by the **attention mechanism**, changed how models handle sequences (especially language).

Attention allowed models to:

- Focus on relevant context
- Learn long-range relationships
- Be trained in parallel more efficiently than RNNs

The important shift: scale becomes king.

A major lesson from the transformer era is:

Many improvements came not from new algorithms, but from **scaling data and computing**.

This era popularised:

- Massive training datasets
- Multi-GPU and multi-node training
- Distributed training infrastructure
- Model scaling as a strategy

Key takeaway:

Transformers made it easier to scale—and scale drove performance.

Milestone 5: Foundation Models and Generative AI

What a foundation model is?

A foundation model is pretrained on massive datasets and then adapted for many tasks through:

- Fine-tuning
- Prompting
- Retrieval augmentation (RAG)
- Tool use/agents

Why has this changed product development?

Historically, you trained a model per task.

Now, you often start with a pretrained model and build systems around it.

This shifts the centre of gravity from:

- “How do we train a model?”
to
- “How do we deploy, control, optimise and govern a model?”

New requirements created are:

Foundation models bring new challenges:

- Higher compute cost at inference
- Latency constraints
- Safety and policy controls
- Monitoring output quality
- Preventing misuse or harmful responses
- Data governance and privacy

Key takeaway:

Modern AI is increasingly about **system design around models**, not training from scratch.

The three major forces that changed AI

1) Data became abundant.

Digital products generate huge amounts of logs, images, text and events.

2) Computing became powerful and accessible.

GPUs and cloud platforms enabled training and serving at scale.

3) Models grew larger and more capable.

Bigger models learned more complex patterns—but also became harder to operate.

Why modern AI problems are engineering problems?

This is the key conclusion for this course:

Modern AI is not primarily a math problem.

It is an **engineering problem**.

Because production AI requires:

- Reliable data pipelines
- Reproducible training
- Evaluation beyond accuracy
- Deployment patterns (CI/CD, canary, rollback)
- Monitoring drift and degradation
- Cost control and performance optimisation
- Governance, privacy and compliance

In short:

Building a model is only one step. Operating the system is the hard part.

Summary

AI evolved from deterministic rule-based systems to data-driven machine learning, then to deep learning and transformers and now to foundation models and generative AI. Each step increased capability—but also increased complexity. As a result, the most complex problems today are not “which model is best,” but how to build reliable systems that can be deployed, monitored, improved, governed and scaled in the real world.